SW Engineering CSC648-848-05 Spring 2023
StudentSources

Team #04 “Fantastic Four”
Saru Shrestha (sshrestha11@sfsu.edu)
Kilian Kistenbroker
Elias Abay
Parth Panchal
Joshua Del Rosario
Wilson Yin
Omar Thiongane

Milestone 4 - Version 2
May 24, 2023

History Table:

M4v2T4	05/24/2023
M4v1T4	04/18/2023
M3v2T4	04/27/2023
M3v1T4	04/15/2023
M2v2T4	04/26/2023
M2v1T4	03/30/2023
M1v2T4	05/24/2023
M1v1T4	03/02/2023

Table Of Contents

Product Summary..... 3
Usability test plan..... 7
QA test plan..... 18
Code Review..... 21
Self-check on best practices for security..... 28
Self-check: Adherence to original Non-functional specs..... 31
List of Team Member Contributions..... 38
Team Member Contribution Score..... 41

Product Summary

Project Name: StudentSources

Committed Functions

1. A general user shall be able to create one or more accounts.
2. A general user shall be able to sign up with a unique email address.
3. A general user shall be able to sign up with a non-unique password.
4. A general user shall be able to see if all requirements for creating a password are met.
5. A general user shall be able to sign up with a non-unique username.
6. A general user shall be able to read the terms and conditions.
7. A general user shall be able to agree to the terms and conditions.
8. A registered user shall have at least one account.
9. A registered user shall be able to view their account information in the Account page.
10. A registered user shall be able to temporarily unhide account password information in the *Account* page. By default, when a registered user visits the *Account* page, the password would be hidden for security and password.
11. A registered user shall be able to change their account username.
12. A registered user shall be able to change their account email.
13. A registered user shall be able to change their account password.
14. A registered user shall be able to upload any supported file type to their account's cloud storage.
15. A registered user shall be able to navigate through files with a directory tree.
16. A registered user shall be able to organize files alphabetically through a directory tree.
17. A registered user shall be able to view the current directory from a *Files* panel.
18. A registered user shall be able to create directories.
19. A registered user shall be able to create text files.
20. A registered user shall be able to create bookmarks of web URLs.
21. A registered user shall be able to pin the resources for easy access in the future.
22. A registered user shall be able to view supported files on the website.
23. A registered user can view the contents of their files on a *Display* panel.

24. A registered user can view the contents of their saved web pages on a *Display* panel.
25. A registered user shall have a place to type and save a note for each resource.
26. A registered user shall be able to comment on files.
27. A registered user shall be able to delete/move resources to a trash bin.
28. A registered user shall be able to delete resources from a trash bin permanently.
29. A registered user shall be able to recover items from a trash bin.
30. A registered user shall be able to download files onto their computer.
31. A registered user shall be able to configure permissions of their shared directories/folders. The permission allows other users to either only view the list or download the list.
32. A registered user shall be able to share their directories by configuring the list as ‘public’.
33. A registered user shall be able to view other directories shared publicly.
34. A registered user shall be able to download other shared directories onto their computer.
35. A registered user shall be able to search public directories.
36. A registered user shall be able to add other registered users as friends.
37. A registered user shall be able to remove other registered users from friend-list.
38. A registered user shall be able to mark other registered users as their favorite friend.
39. A registered user shall be able to configure visibility of their resources at any time. The visibility can be either public or private.
40. A registered user shall be able to message another registered user.
41. A registered user shall be able to create a group chat to message multiple other registered users.
42. A registered user shall be able to rename their resources.
43. Account shall have a certain amount of cloud storage.
44. Accounts shall require and store a username that is at least 6 characters.
45. Accounts shall require and store a real name tied to them, and the real name must have a first and last name.
46. Accounts shall require and store a password tied to them.
47. Accounts password shall be encrypted when stored in a database.
48. Accounts shall be created by one or more general users.
49. An account shall be owned by one registered user.
50. Resources shall be uploaded by a registered user.

51. Resources shall be organized in an alphabetical order.
52. Resources such as text files shall be able to be created by a registered user.
53. Resources shall be viewed by a registered user.
54. Resources shall be commented by registered users.
55. Resources shall be moved by a registered user to a trash bin.
56. Resources shall be able to be deleted permanently by a registered user from a trash bin.
57. Resources shall be recovered by a registered user from a trash bin.
58. Resources shall be downloaded on the computer by a registered user.
59. Resources shall have comments from other registered users.
60. Resources shall be pinned by a registered user.
61. A resource shall have a note created by a registered user.
62. Resources shall have configured visibility by a registered user.
63. Resources shall be able to be renamed by registered users who added them.
64. Directory trees shall be navigated by a registered user.
65. Directory trees shall have files organized in alphabetical order.
66. Directories shall be created by a registered user.
67. Directories shall have configured permissions added by a registered user.
68. Shared directories shall be able to be viewed by a registered user.
69. Directories shall be able to be downloaded by other registered users onto their own computer.
70. Directories of other registered users shall be able to be searched by registered users.
71. Comments shall be added on resources by registered users.
72. A friend shall be added by registered users.
73. A friend shall be removed by registered users.
74. A friend shall be added as a favorite by registered users.
75. A note shall be typed and saved by a registered user for each resource.
76. A registered user shall be able to search other registered users whose profile is set public.
77. A registered user shall be able to search friends.
78. A registered user shall be able to search pinned resources.
79. A registered user shall be able to clear notes added to the resource.
80. A registered user shall be able to clear comments added to the resource.
81. A pinned resource shall be searched by registered users.

82. Comments shall be able to be cleared by a registered user at any time.
83. A friend shall be searched by registered users.
84. Notes shall be cleared by registered users at any time.

Unique Features

We have many unique features that make our website, StudentSources, stand out from the rest. The unique features are as follows: a built-in AI chatbot, a group chat feature, and other ways for students to collaborate such as sharable resources and direct messaging. StudentSources' built-in chatbot is a hot new feature that many websites are adding to enhance the user experience and save the user lots of time when searching for an answer. In addition to the AI chatbot, StudentSources promotes collaboration among students. Through the platform, students can easily share resources, exchange ideas, and collaborate on projects. They can comment on each other's work, direct messages to each other, create a group chat as study group, and have the ability to add or remove friends to stay in touch with users with similar classes and resources. These features make our website an ideal tool for students to enhance their academic experiences and store their study resources.

Product URL: http://54.215.216.18/

Usability test plan

Test Objectives Per Function

1.
Function Being Tested:
A registered user shall be able to search other registered users.

Objective
To determine if users are able to use the search bar with ease and apply filters.

2.
Function Being Tested:
A registered user shall be able to upload any supported file type to their account's cloud storage.

Objective
To determine if users can upload files in a few steps.

3.
Function Being Tested:
A registered user shall be able to delete resources to a trash bin.

Objective
To determine if users find this feature useful in a website for notes organization.

4.
Function Being Tested:
A registered user shall be able to pin the resources for easy access in the future.

Objective
To determine if users find this feature to be efficient.

5.
Function Being Tested:
A note shall be typed and saved by a registered user for each resource.

Objective
To determine users' opinion having a space to add notes for each resource.

Test Description

System Setup

Participants were provided with the website URL so that they are able to access it on your local computer. Testing was performed through physical and virtual presence. In the physical testing environment, the moderator was present in the same room as the participant and observed and recorded the process while a user was navigating through the website. In the virtual environment, participants were requested to share their screen while they carry out the different tasks being tested.

The environment set up required to efficiently perform the test included:

1. Availability of computer devices such as Desktop Computer, Laptop, Tablet or Mobile Phones
2. Availability of Operating Systems such as Microsoft Windows, or MacOS
3. Internet connection to the above mentioned devices either via Ethernet or Wireless connection
4. Availability of web browsers such as Google Chrome, Firefox, Edge, or Safari

Starting Point

The starting point for the test was users were provided with a brief explanation of the website and tasks description. Once users were ready to perform the test, they started from the landing page of the website. If users wished to create an account, they had freedom to do so, otherwise an existing demo account was provided for users to login to the portal. Once, logged in users carried out the tasks listed in the prompt and a moderator recorded all the details. Users were also requested to fill out tables and survey questionnaires to help developers understand user perspective regarding the usability of the website.

Intended Users

The primary targeted users for *StudentSources* were students. Since an effective organization of study notes assists students succeed in their academic goal, further assisting them into their professional career, the participants of the test were students who are in their undergraduate or graduate level studies. While the product is intended for students of any level, we believe that students pursuing higher education would benefit the most from our website. All users participated in the test voluntarily.

The steps for the sample users were as follow:

1. Search
    a. Search other users

2. Upload
    a. Upload a file
3. Pin
    a. Pin a file
    b. Unpin a previously pinned file
4. Delete
    a. Delete a file
5. Notes
    a. Add notes to a file
    b. Clear notes

URL

The website can be accessed using following url:
http://54.215.216.18/

Factors to be measured

With usability testing, we planned to measure the effectiveness and efficiency of the website. Along with the performance factors, we also observed and analyzed the users' satisfaction while using the website.

Usability Test Table: Effectiveness

Test 1

Test/Use Case	% Completed	Errors	Comments
Search	100%	No display of messages when user isn't present	Add display message when no user found
Upload	100%	None	Very fast!
Delete	100%	None	File deleted
Pin	100%	None	It'd be better if users can pin folders from the side panel itself.
Notes	100%	None	No comments

Test 2

Test/Use Case	% Completed	Errors	Comments
Search	100	No errors	A little unintuitive for Student vs Source tab for searching for users
Upload	100	No errors	Very intuitive to standard processes, Drag and drop doesn't work for multiple files, Also doesn't specify what files are accepted for upload, i.e. Similar to how ilearn uploads told users what files were accepted. I was thinking if there could be a more aesthetic approach to Name and Visibility separation with the uploaded files, since everything there has a white background and is right next to each other downwards. Even a

			slight padding to give a separation might help in my opinion.
Delete	100	No errors	Worked fine for 3 vertical dot drop down
Pin	100	No errors	When website is minimized I mistook the wrong pin as pinning the uploaded,
Notes	50	No errors	Clear Notes wasn't working. When adding Notes, for instance clicking on the Text file the middle screen feels as if it's the Notetaking area rather than the right side where it acts not only as the miscellaneous, but also the actual Notes and Comments section.

Test 3

Test/Use Case	% Completed	Errors	Comments
Search	100%	No errors	The search function worked fine
Upload	100%	No errors	Uploading files worked fine
Delete	100%	No errors	Deleting files worked fine but would like to have some more clarity on when clearing the files in the trash (warning before deleting)
Pin	100%	No errors	Pinning files worked perfectly
Notes	100%	- Clear notes button was not working - Same notes were saved	The clear notes button did not work when pressed. The notes are not separate for each individual file.

		across all files	

Test 4

Test/Use Case	% Completed	Errors	Comments
Search	70%	Not implemented	Need to show result
Upload	100%	No error	ok
Delete	80%	Delete but still showing file	Need to delete completely
Pin	80%	Unable to see link	Need to display pin
Notes	80%	No errors also unable to delete note	Need delete completely if asked

Test 5

Test/Use Case	% Completed	Errors	Comments
Search	100%	in some cases, the search results would show ‘to be implemented’	Really liked how the search results will link directly to the file when clicked on.
Upload	100%	No errors	Very quick, although unclear which files are supported.
Delete	100%	No errors	Trash was easy to understand
Pin	100%	No errors	Was expecting pins to always be displayed at the top instead of toggling pins on/off.
Notes	100%	No errors	Notes worked as expected

Usability Test Table: Efficiency

Test/Use Case	Time		Effort	Content / Design	
	Avg. time to complete (in sec)	Avg. time of users who completed / Avg. time of all users (in sec)	Avg. numbers of steps taken to complete (rounded to nearest whole number)	Avg. Number of screens	Number of pages of instructions
Search	64	1.21	3	1	1
upload	5.2	1	3	1	1/3
Delete	12.6	1.15	3	1	1/3
Pin	7.2	1.08	2	1	1/3
Notes	41	1.50	2	1	1/2

User Satisfaction

1.

Function: Search *

Please choose whether you agree or disagree on a scale from 1 (strongly disagree) to 5 (strongly agree)

	1	2	3	4	5
It was easy to use search bar.	○	○	○	○	○
I can use part of a user's name to search their profile.	○	○	○	○	○
The results from search function were satisfactory.	○	○	○	○	○

2.

Function: Upload *

Please choose whether you agree or disagree on a scale from 1 (strongly disagree) to 5 (strongly agree)

	1	2	3	4	5
It was simple to upload a file.	○	○	○	○	○
Uploads take less than a minute.	○	○	○	○	○
I can upload multiple files.	○	○	○	○	○

3.

Function: Comment *

Please choose whether you agree or disagree on a scale from 1 (strongly disagree) to 5 (strongly agree)

	1	2	3	4	5
I was able to add a comment on my file.	○	○	○	○	○
I was able to disable comment.	○	○	○	○	○
I was able to clear comments with few clicks.	○	○	○	○	○

4.

Function: AI Chatbot *

Please choose whether you agree or disagree on a scale from 1 (strongly disagree) to 5 (strongly agree)

	1	2	3	4	5
The chatbot helped me with my needs.	○	○	○	○	○
The chatbot answered fast.	○	○	○	○	○
I liked the feature of the chatbot in this system.	○	○	○	○	○

5.

Function: Notes *

Please choose whether you agree or disagree on a scale from 1 (strongly disagree) to 5 (strongly agree)

	1	2	3	4	5
I can add notes easily.	○	○	○	○	○
I was able to clear notes.	○	○	○	○	○
Notes can be copied.	○	○	○	○	○

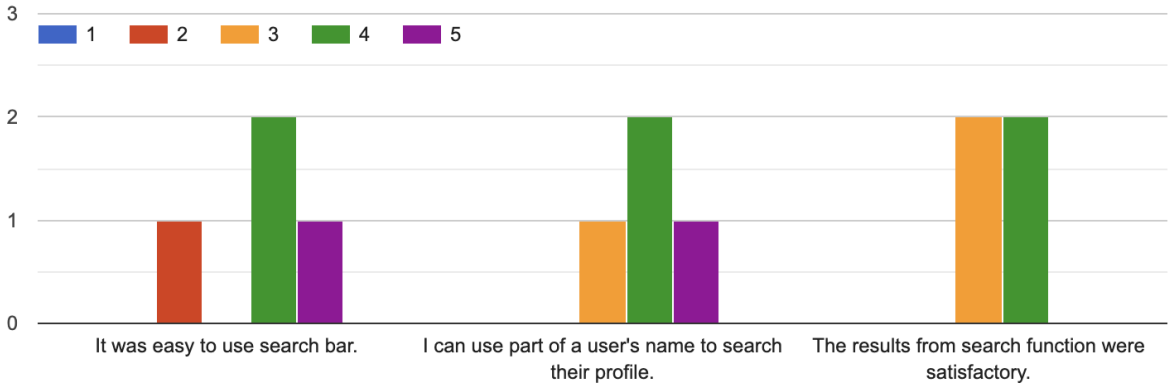
Results of User Satisfaction Evaluation

1.

Function: Search

Copy

Please choose whether you agree or disagree on a scale from 1 (strongly disagree) to 5 (strongly agree)

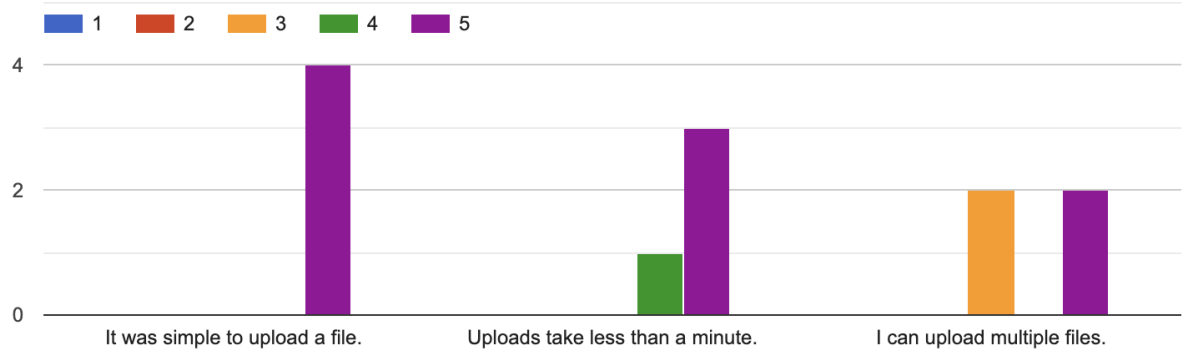


2.

Function: Upload

Copy

Please choose whether you agree or disagree on a scale from 1 (strongly disagree) to 5 (strongly agree)

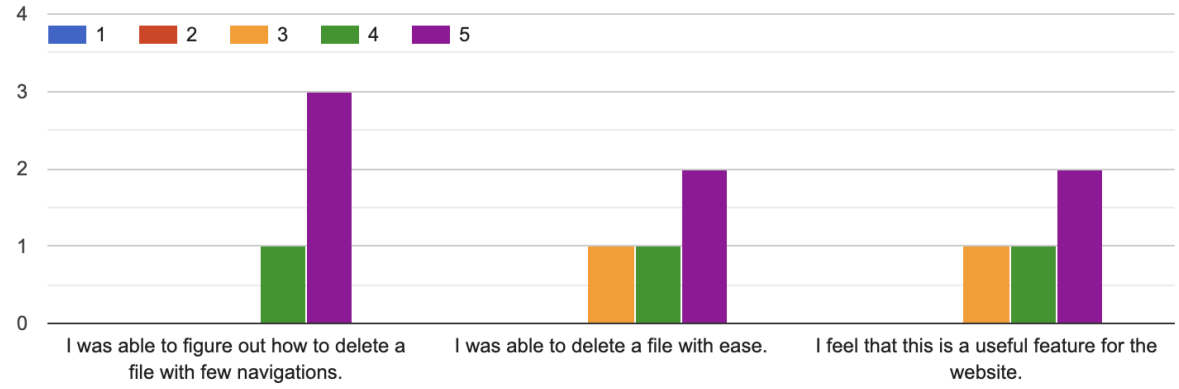


3.

Function: Delete

Copy

Please choose whether you agree or disagree on a scale from 1 (strongly disagree) to 5 (strongly agree)

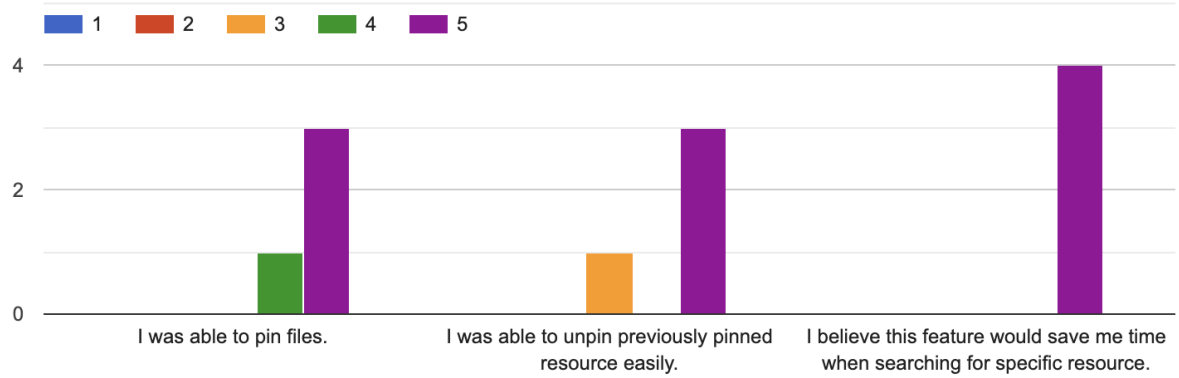


4.

Function: Pin

Copy

Please choose whether you agree or disagree on a scale from 1 (strongly disagree) to 5 (strongly agree)

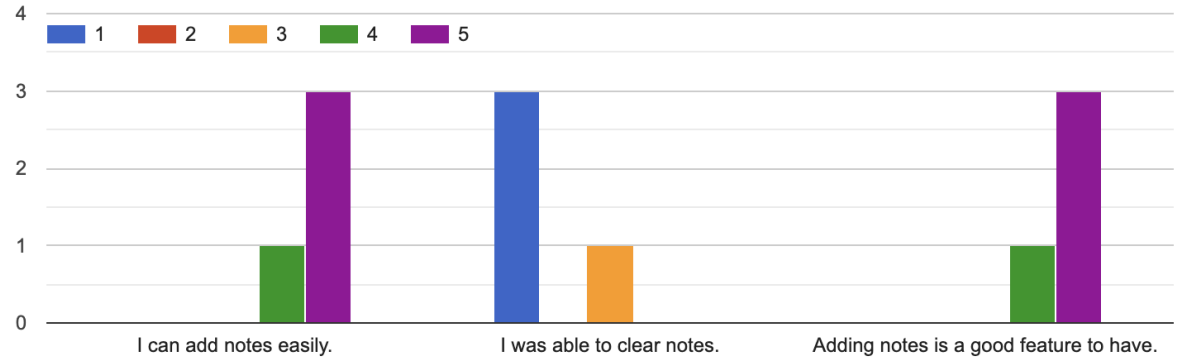


5.

Function: Notes

Copy

Please choose whether you agree or disagree on a scale from 1 (strongly disagree) to 5 (strongly agree)



QA test plan

Test Objectives

The objective of this test is to determine the quality of listed factors:
security - check if the user password is encrypted before being saved to database
storage - ensure that user is aware of the remaining storage capacity
- confirm if the system is able to handle large file sizes
performance - confirm if website is able to handle large number of users simultaneously
compatibility - check is the website is compatible with various web browsers

HW and SW Setup

The hardware setup for this testing were laptops with different operating systems such as Windows, Linux and MacOS. The connection to the wireless WIFI was also secured before the testing began.

The software setup for this test were AWS EC2 instances, Amazon MySQL Relational Database System, Amazon Simple Storage Service. The tests were performed using different web browsers such as Google Chrome, Safari, Mozilla Firefox and Edge.

The software state where testing began were:

1. Password encryption - *Sign up/Login* Page
2. Display storage capacity - *Student* Page
3. Handle large files - *Student* Page
4. Multiple users simultaneously - *Landing* Page
5. Compatible with several web browsers - *Landing* Page

Test Environment

The test environment fulfilled all the requirements of the following checklist:
All required hardware available and connected properly
All required software configured and connected properly
Presence of standard test data sets

In terms of human resources the test required 1 to 2 developers per feature. The expected time to complete testing one feature was less than 1 minute to max 3 minutes.

Risks and Contingencies

1. Late delivery of program code
2. Conflict in the schedule
3. Resources like systems, skills of individuals not properly tracked
4. Failure to resolve responsibilities

Feature to Be Tested

1. System shall always hide registered users account password information on startup the Account page.
2. System shall provide information to registered users the amount of free storage remaining.
3. StudentSources shall be able to handle a large number of simultaneously active users.
4. StudentSources shall be able to handle files with a large size, with the max size being 1 GB.
5. Website shall be compatible with a wide range of web browsers, including but not limited to: Chrome, Firefox, Edge, Safari.

QA Test plan

S.N.	Test Title	Test Description	Test Input	Expected Output	Pass/Fail
1	Security	Check if the user password is being encrypted before storing in the database	Entered password	The password is encrypted	Pass
2	Storage	Ensure that user is aware of the remaining storage capacity	Displayed available storage	The user's remaining storage should be visible	Pass
3	Performance	Check if the website can handle multiple	Multiple testers used website simultaneously	The site should function the same with	Pass

		users simultaneously		multiple concurrent users	
4	Storage	Check if the system can handle large files	Uploaded 30KB	The file should upload	Pass
5	Compatibility	Check if the supports different web browsers	Site opened in Safari, Google Chrome and Mozilla Firefox	The site should open and work similarly without loss of functionality in different browsers.	Pass

Code Review

Coding Style

Our agreed coding style prioritizes the use of proper naming conventions, specifically camelCase for functions and classes. Additionally, code formatting is crucial, with a focus on maintaining consistent indentation and spacing between lines of code. This not only enhances readability but also contributes to an overall neat and organized appearance. Commenting plays a significant role in our coding practices, as we ensure that comments are clear, concise, and directly relevant to the code they accompany. These comments serve as a valuable resource for other team members, allowing them to gain a thorough understanding of the code's purpose and how it has been implemented. By adhering to these coding guidelines, we strive to create code that is not only visually appealing but also facilitates effective collaboration and comprehension among team members.

Code Featuring QA and Usability Test

1.

Code under review

```
package com.example.springbackendv4.controller;

import com.example.springbackendv4.exception.UserNotFoundException;
import com.example.springbackendv4.model.Friends;
import com.example.springbackendv4.repository.FriendsRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@CrossOrigin("http://54.215.216.18:3000/")
public class FriendsController {
    @Autowired
    private FriendsRepository friendsRepository;


    //   ---------------------- BASIC POST, GET, DELETE MAPPING BY UNIQUE ID ---------------------


    @PostMapping("/friend")
    Friends friends(@RequestBody Friends friends){
        return friendsRepository.save(friends);
    }
```

```
    @GetMapping("/friends")
    List<Friends> getAllFriends(){
        return friendsRepository.findAll();
    }

    @GetMapping("/friends/{id}")
    Friends getFriendsById(@PathVariable Long id){
        return  friendsRepository.findById(id)
                .orElseThrow(()-> new UserNotFoundException(id));
    }

    @DeleteMapping("/friends/{id}")
    String deleteFriendsById(@PathVariable Long id){
        if(!friendsRepository.existsById(id)){
            throw new UserNotFoundException(id);
        }

        friendsRepository.deleteById(id);
        return "Deleted friends with id: " + id;
    }



    //   ---------------------- MODIFIED MAPPING (refer to FriendRepository for more info) -------------



    @DeleteMapping("/deleteFriends/{user_id}/{friend_id}")
    String deleteFriends(@PathVariable Long user_id, @PathVariable Long friend_id){
        friendsRepository.deleteFriends(user_id, friend_id);
        friendsRepository.deleteFriends(friend_id, user_id);
        return "Deleted friends : " + user_id + " and " + friend_id;
    }

}
```

Code submission to member of the same team
Review conducted by Omar of Team 4:

A team member implemented code for the "viewing posts" feature, referred to as p1. This work was carried out in a separate feature branch. After completing the code, the members pushed their changes to the feature branch. To incorporate these changes into the main codebase, a pull request was created, providing a description of the modifications made. This pull request was then reviewed by another team member. Following the review process, the changes were merged into the integration branch, also known as the master branch. This integration branch serves as the central repository for combining various feature branches and ensuring the stability and

functionality of the overall codebase. By following this workflow, the team effectively manages code changes, facilitates collaboration and quality control, and ensures that the latest updates are seamlessly integrated into the main codebase for further development and deployment.

The code under review here has proper indentation promoting readability of the code. The presence of comments help understand the functionalities in the code. The name of the file serves its purpose to describe the contents inside it.

Code submission to member of different team

Review conducted by Leo of Team 5:

```
1. Naming Convention  - naming convention is great and
straightforward. I can tell what the file is for and what the
functions do.
2. Files - as mentioned in the naming convention, the file looks
good, very clean.
3. Commenting and Layout - I don't see too much comments, but as a
programmer, I can easily understand the layout.
4. Limited Use of Global Variables and Functions - there's only one
variable and it's private, great!
5. Indentation - formatted nicely and clean, I can see what's
within the class and each functions.
6. Exception Handling - there is error handling in necessary
functions.
7. Statements - great!
```

Reason for choosing this code

This code was chosen because this code implements one of the unique features in the website. Having the snippet of the important file reviewed helps confirm the team if the project is progressing in the right direction.

2.

Code under review

```
package com.example.springbackendv4.controller;

import com.example.springbackendv4.service.StorageService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.core.io.ByteArrayResource;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.multipart.MultipartFile;

import java.util.ArrayList;

/*
* This class will handle requests for file
* upload, download, and deletion from the
* front end. Restrictions should be implemented
* later, so that only signed-in users can
* interact with the aws s3 bucket(s).
*/

@RestController
@RequestMapping("/file")
public class StorageController {

    @Autowired
    private StorageService service;

//    @Autowired
//    private AmazonS3 s3Client;
//
//    @Value("${aws.s3.bucket}")
//    private String bucketName;

    /* This function will handle file upload requests */
    @PostMapping("/upload/{userId}")
    public ResponseEntity<String> uploadFile(@PathVariable Long userId, @RequestParam(value = "file") MultipartFile file) {
        return new ResponseEntity<>(service.uploadFile(userId,file), HttpStatus.OK);
    }
```

```
    /* This function will handle file download requests */
    @GetMapping("/download/{fileName}")
    public ResponseEntity<ByteArrayResource> downloadFile(@PathVariable String fileName) {

//        ----------

        byte[] data = service.downloadFile(fileName);
        ByteArrayResource resource = new ByteArrayResource(data);

        return ResponseEntity
                .ok()
                .contentLength(data.length)
                .header("Content-type", "application/octet-stream")
                .header("Content-disposition, ", "attachment; filename=\"" + fileName + "\"")
                .body(resource);
    }

    /* This function will handle file deletion requests */
    @DeleteMapping("/delete/{fileName}")
    public ResponseEntity<String> deleteFile(@PathVariable String fileName) {
        service.deleteFile(fileName);
        return new ResponseEntity<>("File deleted successfully", HttpStatus.OK);
    }

    @PostMapping("/restore/{fileName}")
    public ResponseEntity<String> restoreFile(@PathVariable String fileName) {
        service.restoreFile(fileName);
        return new ResponseEntity<>("File restored successfully", HttpStatus.OK);
    }

    @PostMapping("/moveFile/{source}/{destination}")
    public ResponseEntity<String> moveFile(@PathVariable String source, @PathVariable String destination) {
        service.moveFile(source,destination);
        return new ResponseEntity<>("File moved successfully", HttpStatus.OK);
    }

    @PostMapping("/moveFolder?source={source}&destination={destination}")
    public ResponseEntity<String> moveFolder(@RequestParam(value = "source") String source, @RequestParam(value = "destination") String destination){
        service.moveFolder(source,destination);
        return new ResponseEntity<>("Folder moved successfully", HttpStatus.OK);
    }

    @DeleteMapping("/trash")
    public ResponseEntity<String> emptyTrash(){
        service.emptyTrash();
        return new ResponseEntity<>("Trash emptied successfully", HttpStatus.OK);
    }
```

```
        /* This endpoint loads the user's saved file directory
        *  The object keys (bucket paths) are returned as an array*/
        @GetMapping("/getUserDir/{userName}")
        public ArrayList<String> listBucket(@PathVariable String userName) {
            return service.getRoot(userName);
        }

        @Scheduled(cron = "0 0 0 * * *")
        public ResponseEntity<String> automaticFileDeletion(){
            service.emptyTrash();
            return new ResponseEntity<>("Trash automatically emptied successfully", HttpStatus.OK);
        }


    }
```

Code submission to member of the same team

Review conducted by Josh of Team 4:

The naming convention for this file/code is consistent with our other files with camelcase for variables, methods, and classes. The literal names for functions are clear and

describe the functions' purpose making the code easy to read and understandable even if someone was to look at this code for the first time. Comments are provided above functions that clearly explain the functions purpose, helping developers who would need to edit the code understand the functionality without having to do extensive research. Layout of the code is appropriate as there is spacing between blocks of code such as functions, to provide readability. The code is also formatted correctly, follows proper indentation practices, well organized, and has imports at the top that are organized.Exception handling is present ensuring any exceptions are caught and handled appropriately. Statements are clear and each function has a specific task. Overall all of these practices contribute to the code's readability and maintainability.

Code submission to member of different team

Review conducted by Johnathan of Team 6:

1. Naming Convention:

The naming convention for the given code snippet is understandable and flows well. Variables are easy to read with camel case naming conventions.

2. Files:

Since their code was written in java, their file name for the code snippet shown is StorageController.java which is the proper java extension. Their main class within this file is StorageController which aligns with their file name which is a good convention to have.

3. Commenting and Layout:

The commenting within the code is very useful to understand what is going on through important functions within the file. Single line comments are also aligned with the given code methods too which is a nice layout.

4. Limited Use of Global Variables and Functions:
I see within the code snippet that there is a limited amount of global variables being used which is good. There are a lot of functions within this code snippet; however, I see the purpose that each function server.

5. Indentation:
Indentation within the code snippet looks nice and formatted, I know there are different methods of indentation; however, overall I can clearly specify where a function starts and ends, and what is being returned within the function–based on the proper indentation done.

6. Exception Handling:

Based on the code snippet, I do not see exception handling for cases which a user wants to upload, delete, or download–i.e no display of message in the case something goes wrong within these methods for the user. This is to be taken with a grain of salt as well because I see these features are only intended for signed-in users.

7. Statement:
Based on the code snippet, I see proper usage of classes and class variable declaration–private variables. I also see no usage of dangerous do-while struct loops which is good.

Signed:
Jonathan Valadez
Team Lead - Team 6

Reason for choosing this code
The reason for choosing the above code is because this file contains functionalities that work with storage. Since configuring a storage system is a significant part of our website, reviewing the above code would verify if it is along the right lines and provide ideas for steps to take moving forward.

Self-check on best practices for security

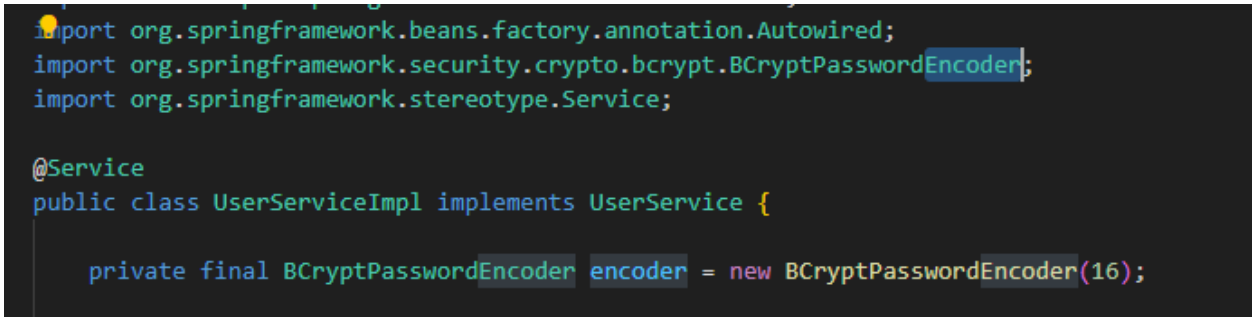
Protected Major Assets

Major Assets we are protecting include:

1. Account Information
    a. First and Last Name
    b. Password
    c. Intellectual Property of Student (uploaded resource)
2. AWS EC2 Instance
    a. SSH Username and Password
    b. SSH Secret Key
    c. SSH URL
3. RDS MySql
    a. Database Hostname
    b. Username and Password to connect to database

Confirmation of Encrypted Password

First, we have imported the BCryptPasswordEncoder framework, which is a PasswordEncoder that uses the BCrypt strong hashing function. We declare a new BCryptPasswordEncoder object with the strength of 16 (log rounds).



```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.stereotype.Service;

@Service
public class UserServiceImpl implements UserService {

    private final BCryptPasswordEncoder encoder = new BCryptPasswordEncoder(16);
```

The encoder is then used in the same file in a function called encryptPasswordAndCreateUser, which will encrypt the password that is typed in by the user during sign in. The encrypted password is then tied to the user and later is stored into the database.

```java
@Override
public String encryptPasswordAndCreateUser(User newUser) throws UsernameAlreadyExistsException, EmailAlreadyExistsException {

    //Check if username is unique
    User userWithUsername = userRepository.findByUser(newUser.getUser());
    if(userWithUsername != null){
        throw new UsernameAlreadyExistsException(newUser.getUser());
    }

    //Check if email is unique
    User userWithEmail = userRepository.findByEmail(newUser.getEmail());
    if(userWithEmail != null){
        throw new EmailAlreadyExistsException(newUser.getEmail());
    }

    String newPassword = encoder.encode(newUser.getPassword());
    newUser.setPassword(newPassword);
    generateToken(newUser);
    User savedUser = userRepository.save(newUser);
    return savedUser.getId().toString();
}
```

Here's an example of passwords being stored in the database:

id	email	first_name	last_name	password	user	storage_left_in_by...
43	omar@yahoo.com	Omar	Thiongane	$2a$16$Wr.9O0yr...	omar	1000000000
42	wilson@yahoo.com	Wilson	Yin	$2a$16$4Z2YAsVz...	wilson	1000000000
41	joshDelRosario@yahoo.com	Joshua	DelRosario	$2a$16$f3Gnaahj....	josh	1000000000
40	saru@yahoo.com	Saru	Shrestha	$2a$16$eHegWnj5...	saru	1000000000
39	elias@yahoo.com	Elias	Abay	$2a$16$Q1m73FE...	eabay42	1000000000
38	kilian@yahoo.com	Kilian	Kistenbroker	$2a$16$Ap4c4J3e...	kilian42	1000000000
37	parthgeek@gmail.com	Parth	Panchal	$2a$16$.bYHt1y8u...	parth	1000000000

Confirmation of Input Data Validation

Input data that is being validated includes:

1. Username
    a. Regular expression is set to ensure the username will be 4 to 24 characters long.

```javascript
const user_regex_l = /\S{4,24}/;
```

2. Password
    a. Three regular expressions are set to ensure:
        i. Password is 8 to 24 characters long
        ii. Password contains at least one letter
        iii. Password contains at least one number

```javascript
const password_regex_l = /\S{8,24}/;
const password_regex_sl = /[a-zA-Z]/;
const password_regex_sn = /[0-9]/;
```

```
<div
  id="pwdnote"
  className={passwordFocus ? "account-instructions" : "offscreen"}
>
  <div className="message_spacing">
    <span className={password_regex_l.test(password) ? "valid" : "hide"}>
      <FontAwesomeIcon icon={faCheck} />
    </span>
    <span
      className={password_regex_l.test(password) ? "hide" : "invalid"}
    >
      <FontAwesomeIcon icon={faTimes} />
    </span>
    Password must contain 8 to 24 characters.
  </div>
  <div className="message_spacing">
    <span className={password_regex_sn.test(password) ? "valid" : "hide"}>
      <FontAwesomeIcon icon={faCheck} />
    </span>
    <span
      className={password_regex_sn.test(password) ? "hide" : "invalid"}
    >
      <FontAwesomeIcon icon={faTimes} />
    </span>
    Password must contain at least 1 number.
  </div>
  <div className="">
    <span className={password_regex_sl.test(password) ? "valid" : "hide"}>
      <FontAwesomeIcon icon={faCheck} />
    </span>
    <span
      className={password_regex_sl.test(password) ? "hide" : "invalid"}
    >
      <FontAwesomeIcon icon={faTimes} />
    </span>
    Password must contain at least 1 letter.
  </div>
</div>
```

3. Email
    a. A regular expression is set to ensure email is

```
const email_regex = /^\w+([-+.']\w+)*@\w+([-.]\w+)*\.\w+([-.]\w+)*$/;
```

4. Search
    a. A regular expression for user search is set to ensure:
        i. Handle username search
        ii. Handle full name search
        iii. Handle search with partial parameter value

```
10 usages  Elias Abay +1 *
public interface RegisteredUserRepository extends JpaRepository<RegisteredUser, Long> {
    1 usage  Elias Abay *
    @Query(value = "SELECT * FROM registered_user WHERE user LIKE :user OR email LIKE :user", nativeQuery = true)
    RegisteredUser findUsers(@Param("user") String user);
```

b. A regular expression for resource search is set to ensure:
   i. Handle folder/files' full name search
   ii. Handle search with partial parameter value

```
public interface UserResourceRepository extends JpaRepository<UserResource, Long> {

    @Query(value = "SELECT * FROM user_resource WHERE visibility LIKE 'Public' AND type LIKE 'Folder'", nativeQuery = true)
    List<UserResource> findAllPublicFiles();


    @Query(value = "SELECT * FROM user_resource WHERE REPLACE(filename, ' ', '') LIKE '%'  :query  '%' AND visibility LIKE 'Publ
    List<UserResource> searchByFilename(@Param("query") String query);
}
```

Self-check: Adherence to original Non-functional specs

S.N.	Non-functional specifications	Progress Report	Remarks
1	System shall always hide registered users account password information on startup the *Account* page.	Completed	Completed
2	System shall periodically delete files from a trash bin.	Completed	Completed
3	System shall provide information to registered users the amount of free storage remaining.	Completed	Completed
4	System shall display an error message when a registered user exceeds the cloud storage limit for their account.	Completed	Completed
5	Any response to StudentSources links shall be within 2 seconds.	Completed	Completed
6	Any response to any StudentSources links shall be within 4 seconds during high volume.	Completed	Did not have an environment to test during high volume.
7	The website shall be able to load text and image files with large sizes within 3 seconds and audio and video with large sizes within 10-15 seconds.	Completed	Completed
8	The website shall be able to load text and image files with large sizes within 5 seconds and audio and	Completed	Completed

	video with large sizes within 20 seconds during high volume.		
9	Log-in and logout of websites shall be almost instant.	Completed	Completed
10	Buttons, textboxes, and options on the landing UI shall be responsive.	Completed	Completed
11	Buttons, textboxes, and options on upload content UI shall be responsive.	Completed	Completed
12	StudentSources shall be able to handle a large number of simultaneously active users.	Completed	Completed
13	StudentSources shall be able to handle files with a large size, with the max size being 1 GB.	Completed	Completed
14	The cloud storage container shall properly store files in TXT format.	Completed	Completed
15	The cloud storage container shall properly store files in PDF format.	Completed	Completed
16	The cloud storage container shall properly store files in MP3 format.	Completed	Completed
17	The cloud storage container shall properly store files in URL format.	Completed	Completed
18	The cloud storage container shall properly store files in JPEG format.	Completed	Completed
19	The cloud storage container shall properly store files in JPG format.	Completed	Completed
20	The cloud storage container shall properly store files in PNG format.	Completed	Completed

21	The cloud storage container shall properly store files in GIF format.	Completed	Completed
22	The cloud storage container shall be able to handle different file formats while still being efficient.	Completed	Completed
23	The database and cloud storage shall store metadata associated with files stored in the cloud storage container.	Completed	Completed
24	Users shall have a registered account to have access to all StudentSources files.	Completed	Completed
25	Passwords shall be encrypted in the database.	Completed	Completed
26	The website shall use secure hosting (AWS).	Completed	Completed
27	The website shall ensure only registered users can access resources.	Completed	Completed
28	The website shall ensure only admins can access sensitive information.	Completed	Completed
29	The website shall be optimized for energy efficiency, optimizing images, using efficient code.	Completed	Completed
30	StudentSources shall encourage students to use electronic files instead of printing and using materials to reduce paper waste and the environmental impact of printing.	Completed	Completed

31	StudentSources shall encourage responsible use by prompting users to delete unwanted files when max capacity of the account is reached to reduce storage and bandwidth of the website.	Completed	Completed
32	StudentSources shall be marketed primarily to students.	Completed	Completed
33	The website shall be marketed as a better way for students to store and organize study resources.	Completed	Completed
34	The website will be marketed as a way to reduce paper waste.	Completed	Completed
35	StudentSources shall comply with copyright laws, to ensure that when the website stores files without infringing the copyrights of others.	Completed	Completed
36	StudentSources shall obtain permission to use copyrighted material according to fair use and copyright laws.	Completed	Completed
37	StudentSources shall have a privacy policy explaining how personal data is collected, used, and protected, which is important for things like student information.	Completed	Completed
38	StudentSources shall have security measures to prevent unauthorized access of data stored in the website’s database.	Completed	Completed
39	StudentSources shall have terms and conditions that govern the use of the website by users.	Completed	Completed

40	Terms and Conditions shall explain rules about uploading and downloading files and acceptable use of the site.	Completed	Completed
41	Content present on the website shall be mostly if not all related to academic courses.	Completed	Completed
42	StudentSources' landing page UI shall be simple for users to understand.	Completed	Completed
43	The website shall be easy to navigate, with users only spending no more than 3-5 minutes browsing around.	Completed	Completed
44	StudentSources UI shall utilize non-distracting colors that complement the content.	Completed	Completed
45	StudentSources UI for uploading content shall be easy to understand and navigate.	Completed	Completed
46	The website shall have a clean and minimalist design,avoiding clutter and aiming for a sleek appearance.	Completed	Completed
47	The website shall have no issues running when experiencing high volume.	Completed	This might depend on the hosting service as well, AWS.
48	The website shall maximize user experience in storing and organizing study resources.	Completed	Completed
49	StudentSources shall only collect data that is necessary for the website's purpose.	Completed	Completed

50	The website shall be accessible using branded computers such as Windows, Apple, Asus.	Completed	Completed
51	The website shall be accessible on a range of devices like desktop computers, laptops, tablets, smartphones.	Completed	Completed
52	Website shall be compatible with operating systems such as Windows and macOS.	Completed	Completed
53	Website shall be compatible with a wide range of web browsers, including but not limited to: Chrome, Firefox, Edge, Safari.	Completed	It was noticed that some functionalities such as uploading multiple files at once via drag and drop behave differently than expected in some browsers
54	The website shall support uploading of a range of file formats, like TXT, PDF, MP3, URL, JPEG, JPG, PNG, GIF.	Completed	Completed
55	StudentSources shall have a user-centered design, focused on ensuring the website is easy to use and navigate for users.	Completed	Completed
56	The website shall have a clear purpose for students and stakeholders and not be too broad.	Completed	Completed
57	The source code shall be clear.	Completed	Completed
58	Source code shall be concise.	Completed	Completed
59	Source code shall be consistent.	Completed	Completed
60	Source code shall be well commented.	Completed	Completed

61	Source code file names shall be clear.	Completed	Completed
62	Source code file names shall be concise.	Completed	Completed
63	Source code file names shall be consistent.	Completed	Completed
64	The file structure of source code shall be easy to navigate.	Completed	Completed
65	Uploaded audio and linked audio and video files shall be streamed to promote better user experience.	Completed	Completed
67	System shall show a registered user a progress and estimated wait time upon uploading files.	Completed	Completed

List of Team Member Contributions

Team Member	**Contribution**
Saru Shrestha	- updated functional and non-functional requirements in all milestone documents - kept track of committed functions and non-functional requirements - reviewed the rough draft of M4 documentation and drafted final draft of the document - recruited sample user for usability test - reached out to members of other teams to review code - reviewed code for Team 4, Team 5 and Team 6 - Revise Milestone 4 documentation
Kilian Kistenbroker	- (implementation of the following) - display user information in the Account panel. (frontend) - provide an option to update/delete information on the Account panel. (frontend) - create a splash message in the bottom right corner for updating, deleting, logging out, and logging in. (frontend) - provide indication of storage remaining on the user account panel. (frontend) - block upload request that exceeds the limit. (frontend) - implement a way to drag folders/files into other folders in the main viewer and directory tree.(frontend) - provide download, rename, and delete options.(frontend/backend) - render items in the trash bin onto the main viewer. (frontend) - configure visibility and permissions. (frontend)

	- Synced comments section and made comments unique per folder and file. (backend) - Synced notes and made notes section unique per folder/file. (backend) - Provided a system for direct/group messaging (frontend/backend) - Made public folders/files searchable. (frontend) - Provided download options when visiting pages where permissions are granted. (frontend) - implemented video download streaming with buffer range. (backend) - Created handling for moving folders/files between directories. (frontend) - Created JSON controller in for managing/manipulating user home directories (backend) - Provided loading progress indicator while files are uploading. (frontend) - sync frontend and backend - recruited sample user for usability test
Elias Abay	- (Implementation of the following - backend) - moving files and folders - generating links for sharing - periodically delete files from a trash bin - recruited sample user for usability test
Parth Panchal	- (Implementation of the following - backend) - handling storage limit - recover files from trash - track progress and estimated wait time after the file is uploaded - typing and saving notes - updating commenting feature to generate unique id and track it in

	database - password encryption when storing in database - recruited sample user for usability test - maintained backup repository
Joshua Del Rosario	- section 1, 5 and 6 of M4 documentation - reviewed code snippet 2
Wilson Yin	- (Implementation of the following - backend) - generate new file or folder after receiving request from frontend to backend - renaming files and folders - downloading directories - viewing shared directories
Omar Thiongane	- section 2 of M4 documentation - reviewed code snippet 1 - recruited sample user for usability test

Team Member Contribution Score

Team Member	Score
Kilian Kistenbroker	10
Elias Abay	9
Parth Panchal	9.5
Joshua Del Rosario	9.5
Wilson Yin	9
Omar Thiongane	9.5

Scale: 1 to 10, where a score of 1 means “no work at all”, and score of 10 means “contributed to the maximum capacity”